Unsupervised Classification of Character Fonts Using Kmeans Clustering

MarcAnthony Cantu

March 2026

Selecting fonts

Among the 5 fonts used in HW1, 3 were chosen to for K-Means Clustering. Those 3 fonts were `CURLZ`, `JOKERMAN`, and `ONYX`. After removing cases with missing numerical data and unnecessary features, {fontVariant, m_label, orientation, m_top, m_left, originalH, originalW, h, w} leaving us just with the features

$font$: True label of the case

strength : A value 0 to 1, indicating normal or bold

italic : A binary value that indicates the style of the font(1=italicized, 0=non-italicized)

$r0c0 - r19c19$: Represents the weight value for each of the 400 pixels in the 20x20 image (values range from 1-255)

The dataset was then filtered to contain **bold_normal** cases where $strength = 0.7$ and $italic = 0$ for all cases. After filtering, the *strength* and *italic* columns were removed and left us with a Nx401 data matrix X with the following number of cases from each font class:

Class	N_i
CURLZ	245
JOKERMAN	247
ONYX	236
Total(N)	728

Table 1: Number of cases from each class

Frequency of each class

The frequency of each class was calculated:

Class	Frequency
CURLZ	%33.65
JOKERMAN	%33.93
ONYX	%32.42

Table 2: Frequency of each class in the dataset

3 cases from each class was randomly selected and displayed:

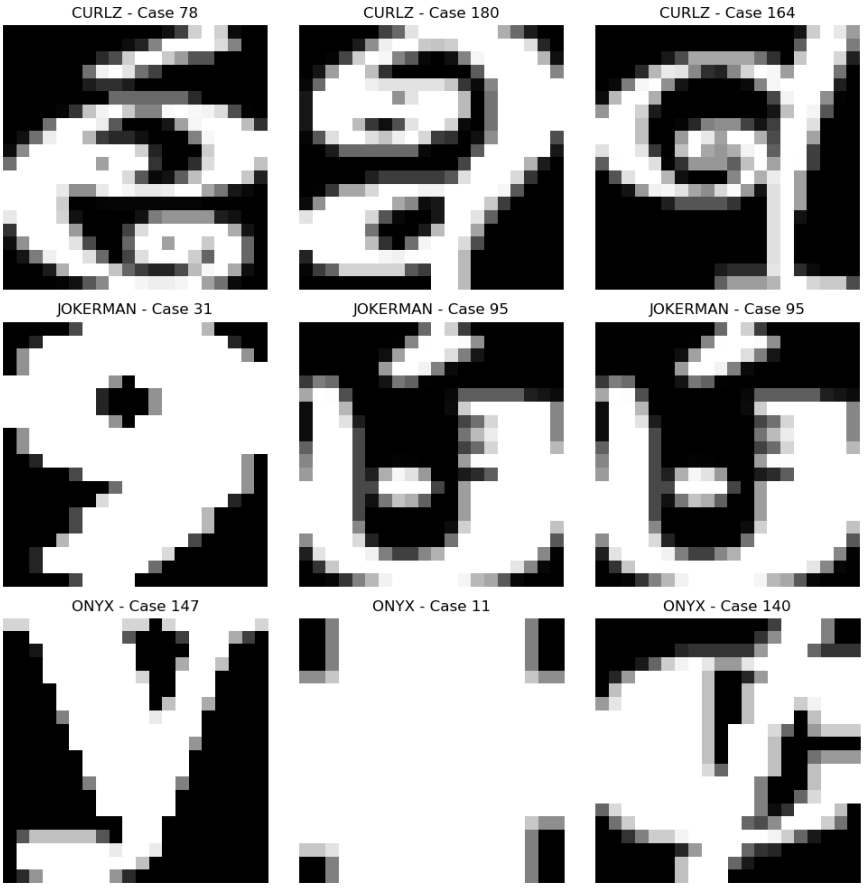


Figure 1: 3 Case Examples from each Class

Correlation of the Data and PCA

The mean μ_i and STD σ_i was computed for each feature $i = 1, \cdots, 400$ and stored in respective vectors $\vec{\mu}$ and $\vec{\sigma}$. A centered and rescaled data matrix Z was then created:

$$Z_{n,i} = \frac{X_{n,i} - \mu_i}{\sigma_i}$$

where n is the n-th row or case of the data matrix. The correlation matrix $Corr(Z)$ was then computed and a new upper triangular matrix $A = |Corr(i,j)|$ where $1 \leq i < j \leq 400$ was created and below displays a histogram of the non-zero, absolute correlation coefficients:

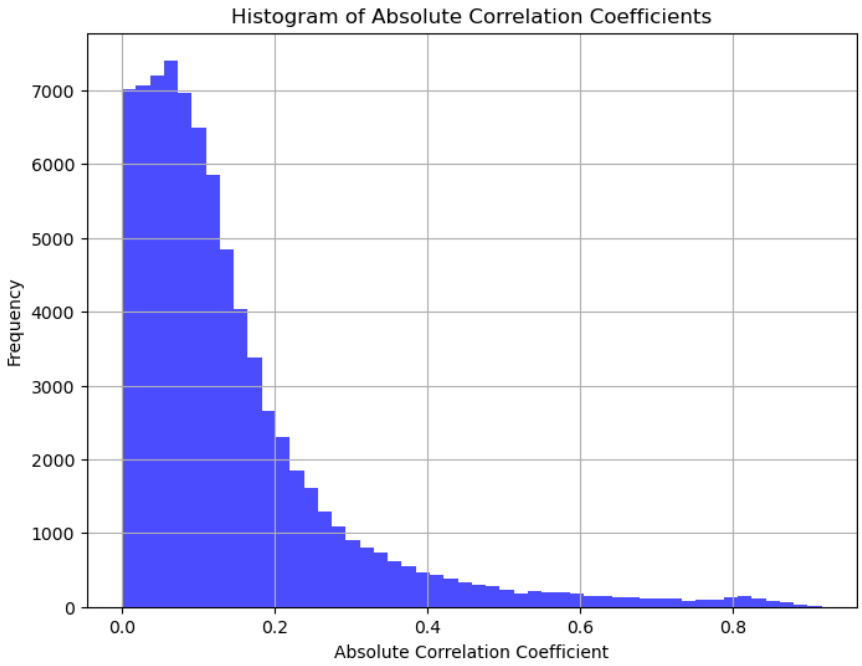


Figure 2: Histogram of Absolute Correlation Coefficients

The top 10 maximum correlation coefficients are located at

i	j	$corr(i,j)$
r19c18	r19c19	0.9155
r12c1	r13c1	0.9108
r0c18	r0c19	0.9092
r0c0	r0c1	0.9072
r0c3	r0c4	0.9047
r10c17	r11c17	0.9046
r15c17	r16c17	0.9045
r0c1	r0c2	0.9042
r9c18	r10c18	0.8951
r11c1	r12c1	0.8932

Table 3: Top 10 largest Correlation Coefficients

The top 10 minimum correlation coefficients are located at:

i	j	$corr(i,j)$
r12c14	r15c8	1.09e-5
r3c13	r8c9	1.17e-5
r2c1	r11c5	1.23e-5
r0c15	r8c16	1.32e-5
r13c19	r16c11	1.35e-5
r2c18	r19c10	1.36e-5
r14c2	r19c1	1.58e-5
r2c19	r18c14	2.50e-5
r7c8	r8c15	2.65e-5
r15c10	r16c19	2.70e-5

Table 4: Top 10 smallest Correlation Coefficients

PCA.
The eigenvalues and eigenvectors of $Corr(Z)$ were computed.

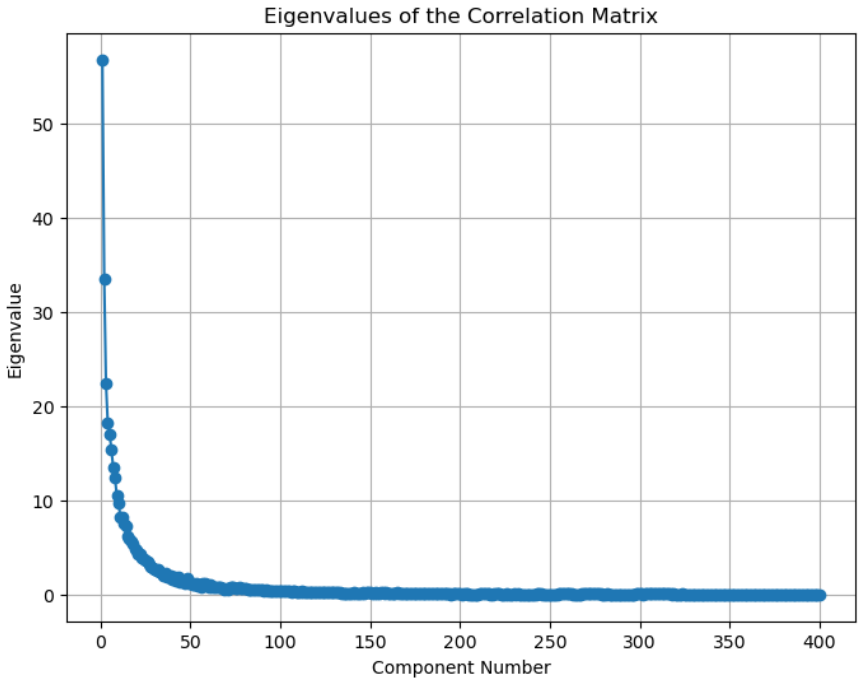


Figure 3: Eigenvalues of Correlation Matrix

The Percentage of Variance Explianed (PVE) gives way to construct a reduced feature dataset U_r that still retains %95 of the variance of X. We do this by first choosing the smallest value r such that $PVE(r) \geq 0.95$

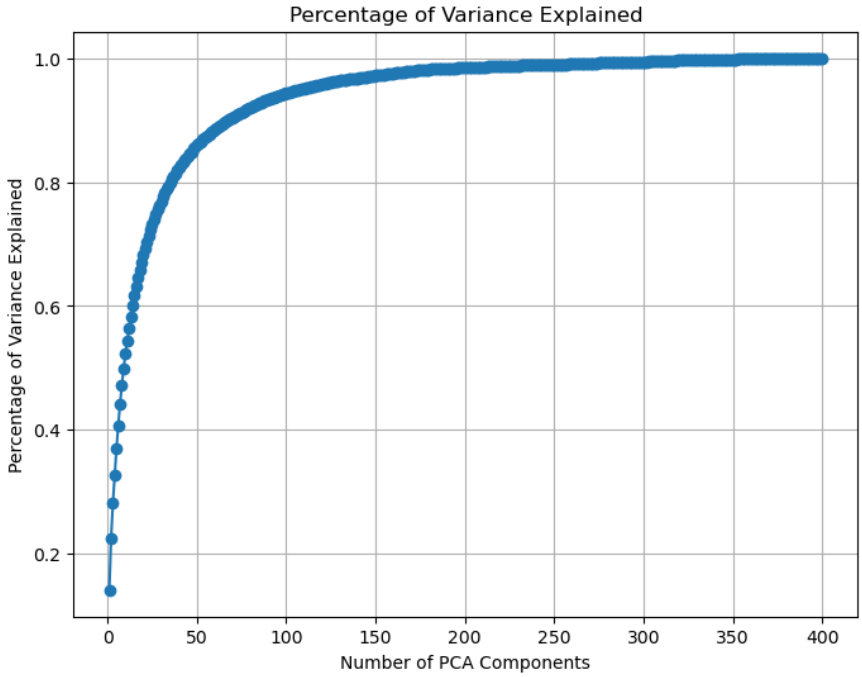


Figure 4: Percentage of Variance Explained

In our case, the desired r value is $r = 109$. We then for the matrix $W_r \in \mathbb{R}^{400\times r}$ that contains the first r eigenvectors and create our reduced data matrix by:

$$U_r = ZW_r \in \mathbb{R}^{N\times r}$$

Plotting the principle components on a scatter plot and labeling each class we get the following display

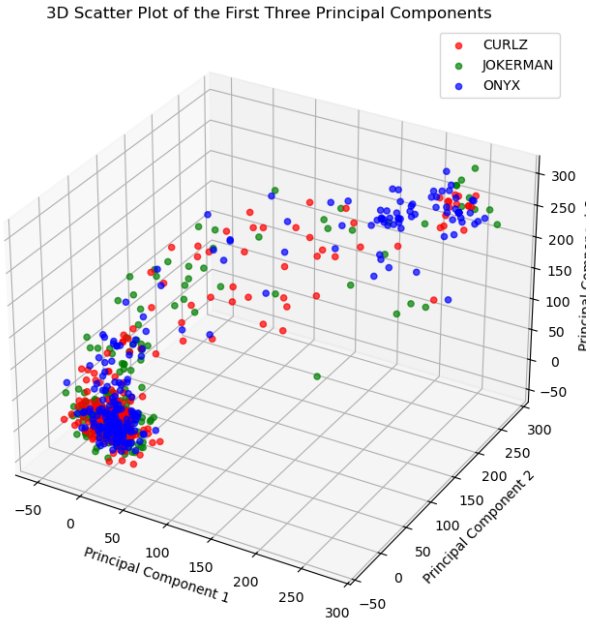


Figure 5: First three principle components for all font cases

From this plot there doesn't appear to be any clear clustering of the cases which may cause difficulties in achieving an accurate clustering

K-Means Algorithm

K-Means algorithm Given a Nx400 data matrix X the K-Means algorithm attempts to find the best suitable clustering of K clusters to the observations by the following steps:

1. Randomly assign each observation x_n to a cluster G_i for $i = 1, \cdots, K$
2. Compute the centroid of each cluster G_i:

$$Centroid(G_i) = C(G_i) = \frac{1}{|G_i|} \sum_{x_n \in G_i} ||x_n||^2$$

3. Reassign each observation x_n to the cluster G_i^* whose centroid is closest to x_n
4. iterate over steps 2 and 3 until a local minimum is found

Total Dispersion The dispersion of a cluster G_i is a way of measuring the disparity of the or "spread" of the data in cluster G_i and is given by:

$$disp(G_i) = \sum_{x_n \in G_i} ||x_n - C(G_i)||^2$$

The total dispersion gives us a measurement of the overall quality of a clustering:

$$totalDISP(G) = \sum_{i=1}^{K} disp(G_i)$$

Hyperplane A hyperplane H separating 2 clusters G_i and G_j is the set of all observations x^* such that

$$||x^* - C(G_i)||^2 = ||x^* - C(G_j)||^2$$

Given that our data matrix has 400 feature vectors, our hyperplane can be expressed as a linear equation in dimension 400 of x^*:

$$u_1 x_1^* + u_2 x_2^* + \cdots + u_{400} x_{400}^* = b$$

where

$$U = [u_1, u_2, \cdots, u_{400}] = 2(C(G_i) - C(G_j))$$

and

$$b = ||C(G_i)||^2 - ||C(G_j)||^2$$

Scikit-Learn KMeans

1. Overview

The `KMeans` class implements the standard k-means clustering algorithm. Given a dataset $X \in \mathbb{R}^{N \times p}$, the algorithm partitions the samples into k clusters by minimizing the within-cluster sum of squared Euclidean distances.

2. Inputs to the `KMeans` Methods

There are two input levels: the data provided to the methods, and the meta-parameters provided when constructing the estimator.

2.1 Data Inputs

- **X**: array-like matrix of shape (N, p). This is the dataset to be clustered or transformed.
- **sample_weight**: optional array-like of shape N assigning a weight to each point.

Method-specific inputs and outputs:

- `fit(X, y=None, sample_weight=None)`
  - Input: X, optional sample weights.
  - Output: the fitted estimator (self).
- `fit_predict(X, y=None, sample_weight=None)`
  - Output: label vector of shape N with cluster indices.
- `predict(X)`
  - Output: cluster labels for new data.
- `fit_transform(X)`
  - Output: distance matrix of shape (N, K).
- `transform(X)`
  - Output: distances to each cluster center.
- `score(X)`
  - Output: Opposite of the value of X on the K-means objective..

3. Outputs After Fitting

Once `fit` is called, the estimator stores:

- **cluster_centers_**: array of shape (k, n_{features}) containing the final centroids.
- **labels_**: array of shape (n_{samples}) with the cluster assignment for each point.
- **inertia_**: the within-cluster sum of squared distances.
- **n_iter_**: number of iterations completed.
- **n_features_in_**, **feature_names_in_**: metadata describing the input features.

4. Meta-Parameters (Hyperparameters)

The meta-parameters below are set when constructing the estimator:

```
KMeans(
    n_clusters=8,
    init="k-means++",
    n_init="auto",
    max_iter=300,
    tol=1e-4,
    verbose=0,
    random_state=None,
    copy_x=True,
    algorithm="lloyd"
)
```

4.1 `n_clusters` (int, default=8)

- Number of clusters k.
- Larger k captures more structure but risks overfitting/noise.
- Smaller k may merge distinct groups.

4.2 `init` (`"k-means++"`, `"random"`, array-like, or callable)

- Determines how initial centers are selected.
- Options:
  - `"k-means++"` (default): smart initialization that improves convergence.
  - `"random"`: random points as initial centers.
  - Array-like: user-specified initial centroid set.
  - Callable: custom initialization strategy.
- Strong influence on convergence and risk of poor local minima.

4.3 `n_init` ("auto" or int, default="auto")

- Number of independent runs with different initializations.
- Best solution (lowest inertia) is retained.
- Increasing this reduces the probability of getting a poor local optimum.

4.4 `max_iter` (int, default=300)

- Maximum number of Lloyd iterations per run.
- Too small a value may prevent convergence.

4.5 `tol` (float, default=1e-4)

- Convergence tolerance threshold.
- Smaller values give more precise but slower convergence.

4.6 `verbose` (int, default=0)

- Controls output of logging information.
- No effect on model quality.

4.7 `random_state` (int or None)

- Seeds the random number generator for reproducibility.

4.8 `copy_x` (bool, default=True)

- If True, X is copied before internal transformations.
- If False, X may be temporarily modified.

4.9 `algorithm` (`"lloyd"` or `"elkan"`, default="lloyd")

- `"lloyd"`: standard k-means algorithm.
- `"elkan"`: uses triangle inequality to skip distance computations; faster on low-dimensional data.

KMeans Clustering for $K = 1, \cdots, 20$

K clusters were generated for $K = 1, \cdots, 20$. For each value of K 100 runs were implemented with random initializations and the run with the smallest total dispersion was stored. The performance of each clustering is evaluated by:

$$perf(K) = \frac{totalDIPS(K)}{totalDISP(1)}$$

Where $totalDIPS(1)$ represents the dispersion of the entire dataset. For each generation of K-clusters, the computational time was also recorded:

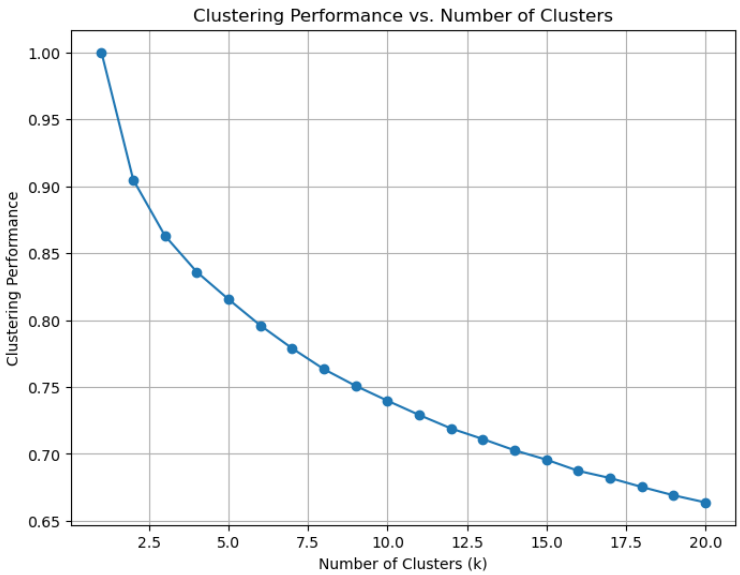


Figure 6: Performance per K-Clustering

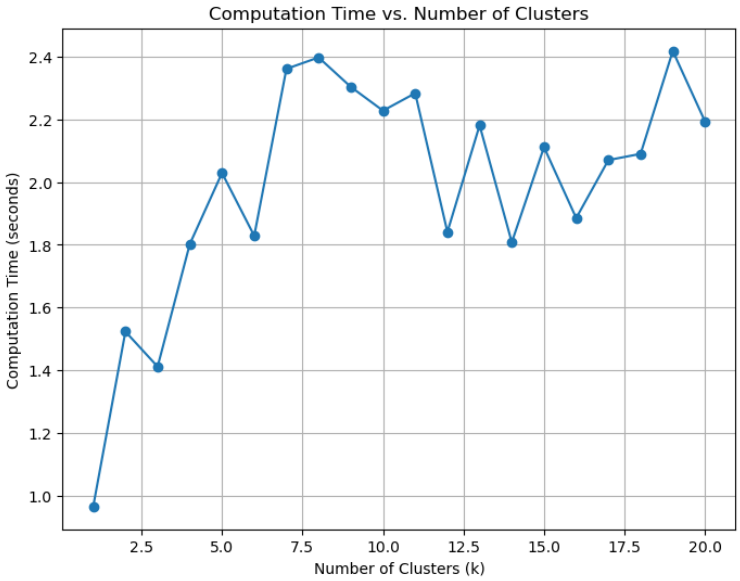


Figure 7: Computation time per K-Clustering

We can see from these graphs that the performance curve decreases and the computation time increases sporadically as K increases. I would also consider the computation time as non-negligible since it appears to increase relatively quickly.

The Elbow Rule and Selecting an Optimal K^*

The elbow rule is a method of finding an optimal point of diminishing returns. In the context of K-Means clustering, the rule suggests that we choose a value K such that for all $k > K$, $perf(K) - perf(k)$ becomes negligibly small. This point on the performance curve will appear similar to an "elbow" or "joint" in the curve.
Based on our performance curve and the elbow rule, $K^* = 3$ appears to be this point. The performance for the K^*-clustering is

$$perf(3) = 0.8629$$

Our clustering G is composed of 3 clusters $G = [G_1, G_2, G_3]$ Each with the following properties:

G_i	Relative size	Relative dispersion	Radius
G_1	0.3049	0.2831	1989.81
G_2	0.2898	0.2815	2033.98
G_3	0.4052	0.4355	2139.21

Table 5: Cluster data

Cluster Centroids and Separability of Clusters

The centroid g_i for each cluster G_i was computed and stored in a csv file `Centroid.csv`. Given this data, we found the sparability of each pairing of clusters by

$$sep(G_i, G_j) = \frac{||g_i - g_j||}{rad(G_i) + rad(G_j)}$$

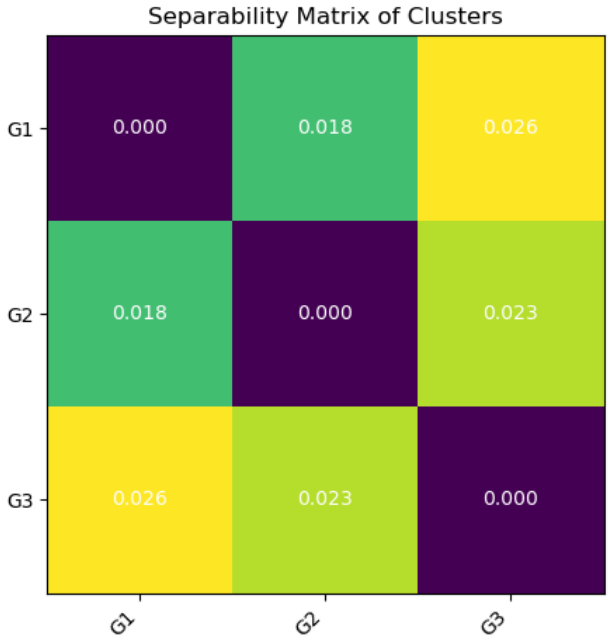


Figure 8: Separability Matrix

Based on these results, our clusters have very low separability which implies ill-suitability for our data.

Frequency of Clusters among Font Classes

We define the frequency of set A in set B as

$$freq(A;B) = \frac{|A \cap B|}{|B|}$$

Using this, we calculate the frequency of each class $CL = \{\text{CURLZ, JOKERMAN, ONYX}\}$ in each of our clusters G_i as well as the purity of each cluster.

	G_1	G_2	G_3
CURLZ	12.9%	17.8%	58.5%
JOKERMAN	33.2%	42.6%	26.9%
ONYX	46.6%	39.1%	14.3%
Purity	41.6%	37.1%	43.8%

Table 6: Frequency and Purity of each Cluster

The cluster with Highest purity is G_3 with purity 43.8% and the highest frequency among our classes with respect to g_3 is $CL1 = \text{CURLZ}$ with frequency 58.5%
Analyzing the set differences $Cl_1 - G_3$ and $G_3 - CL_1$ we obtain sets of size:

$$|Cl_1 - G_3| = 66$$
$$|G_3 - CL_1| = 121$$

So, we can see that G_3 contains a large majority of CL_1 while CL_1 contains a little over half of G_3

High Interest Clustering

Here we treat our classes $CL = \{\text{CURLZ, JOKERMAN, ONYX}\}$ as a "high interest clustering" (HIC). The relative dispersions of each class, total dispersion, and performance of CL is shown in the table below:

Class	Relative Dispersion
CURLZ	1.18×10^9
JOKERMAN	1.13×10^9
ONYX	1.13×10^9
$totalDIPS(HIC)$	3.45×10^9
$perf(HIC)$	0.9617

Table 7: Data of High Interest Clustering

Comparing HIC and K^*-Clustering

Comparison of the performance of HIC and G

$$perf(G) = 0.8629$$
$$perf(HIC) = 0.9617$$

We can see that the total dispersion of HIC is very close to the total dispersion of the whole dataset and varies from $perf(G)$ by 0.0988. Since K-Means clustering seeks to find a cluster of minimal total dispersion, it will have difficulty finding out high interest cluster.
Based on our frequency table from Q8, the classes from CL and clusters G that are most similar are:

$$CL_1 = \text{CURLZ} \sim G_3$$
$$CL_2 = \text{JOKERMAN} \sim G_2$$
$$CL_3 = \text{ONYX} \sim G_1$$

Thus, our new permutation for G is $G^* = [G_{31}, G_{22}, G_{13}]$.

The accuracy of this permuted cluster is given by:

$$\begin{aligned} acc &= \frac{\sum_{i=1}^{3}(freq(CL_i; G_{i*})|CL_i|}{N} \\ &= \frac{0.585(245) + 0.426(247) + 0.466(236}{728} \\ &= \boxed{0.4925} \end{aligned}$$

Thus our clusters has a roughly 50% accuracy.

Directed K-Means Clustering

This time we will run KMeans with the initialization being the centroids of our classes.

```
U_Clusters = KMeans(n_clusters=3, n_init=100, init=HIC_centroids).fit(X)
```

Doing so generates a new cluster $U = [U_1, U_2, U_3]$ with a performance value of

$$perf(U) = 0.8630$$

The performance of U is very close to the performance of G with only a 0001 difference. Computing the frequency matrix we can see that the highest frequencies are along the diagonal:

Thus, the class to cluster pairing is one-to-one and our permutation is $U^* = [U_{11}, U_{22}, U_{33}]$ with accuracy

$$Acc_U = 0.5005$$

The improvement of U in comparison to G is almost negligible.

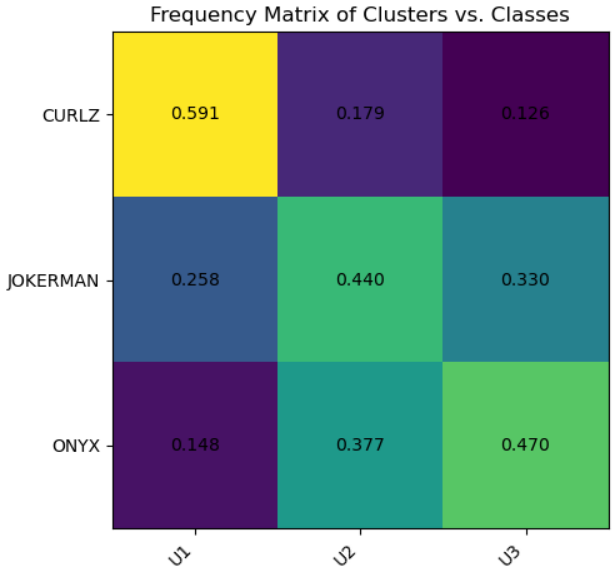


Figure 9: U Frequency Matrix